**COMSATS University Islamabad, Vehari Campus, Pakistan**

Department of Computer Science

# Smart Blood Bank Mobile Application

A Final Year Project Report Submitted in Partial Fulfillment of the Requirements
for the Degree of

## Bachelor of Science in Computer Science

**Submitted By**

Muhammad Usman (CIIT/SP23-BCS-046/VHR)
Muhammad Hassan (CIIT/SP23-BCS-038/VHR)

**Supervisor**

Sir Najeeb Ullah Khan

Session: Spring 2023–2027

# Dedication

I dedicate this work to my parents, whose unwavering support, sacrifices, and prayers have been the foundation of my success. Their encouragement, patience, and belief in my abilities have guided me throughout this journey.

I also dedicate this to my teachers and mentors who inspired me to pursue knowledge with dedication and integrity.

# Acknowledgement

All praise and gratitude are due for the successful completion of this project. The authors would like to express their sincere appreciation to their supervisor for continuous guidance, valuable suggestions, and constructive feedback throughout the development of this thesis. The supervision and mentorship provided significant support in completing this work successfully.

Special thanks are extended to the faculty members of the Department of Computer Science for their academic support and encouragement during the degree program.

The authors are also deeply grateful to their parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength.

Finally, appreciation is extended to everyone who directly or indirectly contributed to the successful completion of this project.

# Certificate of Approval

This is to certify that the Final Year Project titled **"Smart Blood Bank Mobile Application"** has been developed by **Muhammad Usman (CIIT/SP23-BCS-046/VHR)** and **Muhammad Hassan (CIIT/SP23-BCS-038/VHR)** under the supervision of **Sir Najeeb Ullah Khan** and that in his opinion, the project is adequate in scope and quality for the award of the degree of Bachelor of Science in Computer Science.

---

Supervisor
Sir Najeeb Ullah Khan
Department of Computer Science
COMSATS University Islamabad, Vehari Campus

---

External Examiner

---

Head of Department
Department of Computer Science

# Plagiarism and AI Content Certificate

Date: _____ It is hereby certified that the similarity index generated by the

Turnitin Originality Report for the **Final Year Project Report** submitted by:

- **Muhammad Usman (CIIT/SP23-BCS-046/VHR)**
- **Muhammad Hassan (CIIT/SP23-BCS-038/VHR)**

titled:

**"Smart Blood Bank Mobile Application"**

is [**Similarity Index**] which must be completed from the official originality report before submission. No percentage is asserted here without verified evidence. AI-generated content detection, if

required by institutional policy, should likewise be completed from the official report. **Supervi-**

**sor Name:** Sir Najeeb Ullah Khan **Designation:** _____ **Signature:**

_____ **Department:** Department of Computer Science **University:**

COMSATS University Islamabad, Vehari Campus

# Executive Summary

Smart Blood Bank is a Flutter and Firebase application intended to improve coordination between voluntary blood donors, blood receivers, and system administrators. Conventional coordination through telephone calls and social-media messages is often unstructured, exposes personal information broadly, and provides limited support for compatibility, distance, availability, progress tracking, or accountability.

The system provides seven connected modules: Authentication, Donor, Receiver, Notification, Reward and Certificate, Admin, and Mapping and Distance. Users register through Firebase Authentication and receive role-specific navigation. Receivers can create structured, multi-unit blood requests and emergency SOS alerts. Donors can manage availability and location consent, discover compatible requests, accept assignments, indicate readiness, and review donation history and eligibility information. Confirmed donations contribute points and can be represented by generated PDF participation certificates. Administrators use a role-protected dashboard for monitoring, user-status actions, reports, misuse handling, statistics, exports, and audit-oriented operations.

Sensitive workflow transitions are implemented in callable Cloud Functions rather than relying on unrestricted client writes. Firestore and Storage rules enforce ownership, participation, active-account, and administrator checks. Nearby discovery combines blood-group compatibility, account and eligibility policy, location consent, an indexed geographic pre-filter, and Haversine distance. Push notifications, phone OTP, certificate storage, and route-time estimates remain dependent on deployed services, platform permissions, and network availability.

The project is a coordination platform and not a clinical blood-bank information system. It does not perform laboratory screening, cross-matching, diagnosis, inventory management, or guaranteed emergency response. The implementation demonstrates how a responsive mobile interface and serverless backend can improve structure, privacy, traceability, and donor engagement while preserving appropriate clinical boundaries.

# Abbreviations

| Abbreviation | Full Form |
|---|---|
| **SRS** | Software Requirements Specification |
| **SDD** | Software Design Document |
| **API** | Application Programming Interface |
| **UI** | User Interface |
| **DB** | Database |
| **FCM** | Firebase Cloud Messaging |
| **GPS** | Global Positioning System |
| **OTP** | One-Time Password |
| **OSRM** | Open Source Routing Machine |
| **SOS** | Emergency Distress Alert |
| **UML** | Unified Modeling Language |

# Table of Contents


Executive Summary . . . . . v
List of Tables . . . . . ix
List of Figures . . . . . x

**1 Introduction** **1**
1.1 Background . . . . . 1
1.2 Problem Statement and Motivation . . . . . 1
1.3 Aim and Objectives . . . . . 1
1.4 Proposed Solution . . . . . 1
1.5 Scope and Limitations . . . . . 1
1.6 Significance . . . . . 2
1.7 Thesis Organization . . . . . 2

**2 Literature Review** **3**
2.1 Blood Donation and Digital Coordination . . . . . 3
2.2 Comparison of Approaches . . . . . 3
2.3 Gap and Contribution . . . . . 3
2.4 Chapter Summary . . . . . 3

**3 System Analysis and Requirements** **4**
3.1 Roles . . . . . 4
3.2 Seven Modules and Verified Scope . . . . . 4
3.3 Functional Requirements . . . . . 5
3.4 Use Cases . . . . . 5
3.5 Non-functional Requirements . . . . . 5
3.6 Constraints, Assumptions and Feasibility . . . . . 6

**4 System Design** **7**
4.1 Overall Architecture . . . . . 7
4.2 Interaction and Workflows . . . . . 7
4.3 Database Design . . . . . 8
4.4 UML-style Views . . . . . 8
4.5 Security and Data Flow . . . . . 8
4.6 Chapter Summary . . . . . 9

**5 Implementation** **10**
5.1 Implementation Environment . . . . . 10
5.2 Authentication Module . . . . . 10

5.3 Donor Module . . . 12
5.4 Receiver Module . . . 14
5.5 Notification Module . . . 15
5.6 Reward and Certificate Module . . . 16
5.7 Admin Module . . . 16
5.8 Mapping and Distance Module . . . 18
5.9 Implementation Outcome . . . 18

**6 Software Testing** **20**
6.1 Objectives, Scope and Integrity . . . 20
6.2 Approach and Environment . . . 20
6.3 Module Assessments . . . 23
6.3.1 Authentication Module . . . 23
6.3.2 Donor Module . . . 23
6.3.3 Receiver Module . . . 23
6.3.4 Notification Module . . . 23
6.3.5 Reward and Certificate Module . . . 23
6.3.6 Admin Module . . . 23
6.3.7 Mapping and Distance Module . . . 23
6.4 Visual Test Evidence . . . 23
6.5 Security, Limitations and Summary . . . 27

**7 Conclusion and Future Work** **28**
7.1 Conclusion . . . 28
7.2 Future Work . . . 28
7.3 Limitations . . . 28

**Appendices** **30**
Appendix A – Turnitin Similarity Report . . . 30
Appendix B – AI Writing Detection Report . . . 31
Appendix C – Source Code . . . 32

# List of Tables

2.1 Related approaches and gaps . . . . . 3

3.2 Representative use cases . . . . . 5

4.1 Principal Firestore collections . . . . . 8

6.1 Module test cases and evidence . . . . . 21

# List of Figures


4.1 Implemented high-level architecture . . . . . 7
4.2 Request-creation sequence . . . . . 8

5.1 Login screen with email/password fields and password-reset access . . . . . 11
5.2 Create-account screen with profile, role and blood-group fields . . . . . 12
5.3 Donor dashboard showing donation summary, eligibility and quick actions . . . . . 13
5.4 Donor profile showing personal information and blood-group details . . . . . 14
5.5 Administrative notification composition and history interface . . . . . 15
5.6 Administrative dashboard with summary cards and recent activity . . . . . 17
5.7 Administrative user-management view with approved users and role filters . . . . 17
5.8 Administrative audit and export view showing an audit event . . . . . 18

6.1 Role-selection screen used to inspect donor and receiver navigation entry points. . 24
6.2 Settings screen used to inspect profile data, notification controls and donation availability. . . . . 25
6.3 Reset-password dialog used to inspect input presentation and the reset-link action. 26
6.4 Admin audit view used to inspect audit-log visibility and export controls. . . . . 26

# Chapter 1
# Introduction

## 1.1 Background

Timely access to compatible blood depends on coordination among recipients, voluntary donors, hospitals, and administrators. WHO identifies voluntary donation, donor selection, testing, and quality systems as foundations of safe blood services [1, 2]. A digital system cannot replace clinical screening, but it can reduce communication delay and improve traceability.

Smart Blood Bank is a Flutter and Firebase coordination application. It combines accounts, requests, SOS alerts, compatibility and distance-based discovery, donation records, rewards, certificates, notifications, and administration. It does not test blood, diagnose patients, guarantee availability, operate an ambulance, or maintain hospital inventory.

## 1.2 Problem Statement and Motivation

Calls and social-media appeals provide weak identity control, broad disclosure, limited location awareness, and no durable audit trail. Receivers may contact incompatible, unavailable, distant, or temporarily ineligible donors. Mobile applications can support donor engagement, but usability, trust, privacy, and deployment quality matter [3]. This motivates structured requests, consent-aware location, server-authoritative transitions, and accountable administration.

## 1.3 Aim and Objectives

The aim is to implement a secure and usable platform for donor–receiver coordination. Objectives are authenticated role access; structured requests and SOS; compatible, eligible, available and nearby discovery; assignment and donation tracking; history, points and certificates; deployable notifications; role-protected administration; and privacy-preserving, auditable data access.

## 1.4 Proposed Solution

Flutter provides role-specific clients. Firebase Authentication establishes identity; Firestore stores data; callable Cloud Functions validate sensitive workflows; Cloud Messaging supports notifications; and Storage holds generated certificates. Geolocation and OpenStreetMap maps support discovery, while OSRM may supply driving-time estimates.

## 1.5 Scope and Limitations

Implemented scope includes email/password access, optional verification flows, profiles, requests, SOS linkage, matching, confirmation, history, eligibility dates, points, PDF certificates, notifications, mapping, and a dashboard. Phone OTP, push delivery, functions, storage, routing, and

permissions require configured services and device testing. New profiles are `approved`; login blocks only `suspended` and `rejected`. Clinical screening, laboratory cross-matching, inventory, payment, chat, and guaranteed emergency response are excluded.

## 1.6 Significance

The project demonstrates mobile coordination with explicit clinical boundaries, least-privilege rules, event history, controlled contact access, and opt-in location sharing.

## 1.7 Thesis Organization

Chapter 2 reviews related work; Chapter 3 defines requirements; Chapter 4 presents design; Chapter 5 documents implementation; and Chapter 6 reports test evidence.

# Chapter 2
# Literature Review

## 2.1 Blood Donation and Digital Coordination

WHO guidance places suitability assessment, infection screening, storage, and clinical governance within regulated services [1, 2]. Coordination software must help communication without posing as a clinical decision system. A systematic review reports that blood-donation apps commonly use reminders, education, appointments, location, and urgent appeals, while evidence quality and sustained engagement vary [3]. The American Red Cross app illustrates an integrated donor journey covering appointments, history, eligibility information, and rewards [4].

## 2.2 Comparison of Approaches

Table 2.1: Related approaches and gaps

| Approach | Strengths | Limitations | Project response |
|---|---|---|---|
| Calls/social media | Immediate and familiar | Unstructured, excessive disclosure, no audit trail | Structured requests and controlled access |
| Institutional app | Trusted appointments and donor history | Organization/region specific | Local donor–receiver coordination |
| Map directory | Fast proximity search | May ignore eligibility, consent and compatibility | Backend policy plus distance filtering |
| Smart Blood Bank | Integrated workflow and administration | Requires deployed cloud services; non-clinical | Explicit boundaries and graceful errors |

## 2.3 Gap and Contribution

The gap is not merely displaying donor phone numbers. A practical prototype must combine identity, compatibility, eligibility, consent, distance, multi-unit state, controlled contact disclosure, duplicate-safe confirmation, notifications, and auditability. Smart Blood Bank uses callable workflows, latitude pre-filtering plus Haversine distance, and stable donation identifiers. Rewards and certificates derive from confirmed donations. This is an engineering contribution, not proof of improved clinical outcomes.

## 2.4 Chapter Summary

Prior work supports mobile engagement and location-aware coordination while emphasizing trust, privacy, clinical boundaries, and rigorous evaluation. These findings inform Chapter 3.

# Chapter 3
# System Analysis and Requirements

## 3.1 Roles

The **donor** manages profile, consent and availability, accepts requests, records readiness, and views history/rewards. The **receiver** creates and tracks requests/SOS and confirms donations. The **administrator** uses a role-protected dashboard for users, requests, reports, misuse handling, statistics, exports, and audits.

## 3.2 Seven Modules and Verified Scope

| Module | Responsibility | Status |
|---|---|---|
| Authentication Module | Registration/login, optional email/phone verification, validation, reset and role routing. | Implemented; OTP deployment/device dependent. |
| Donor Module | Profiles, availability, history, eligibility, rewards and certificates. | Implemented; clinical eligibility external. |
| Receiver Module | Multi-unit requests, tracking, cancellation, confirmation and SOS. | Implemented through callables. |
| Notification Module | In-app and push alerts for requests, eligibility and drives. | Implemented; push needs deployed FCM/functions. |
| Reward and Certificate Module | Points and PDF acknowledgement certificates. | Implemented; no public QR verifier. |
| Admin Module | Monitoring, status actions, reports, misuse, statistics and audit. | Implemented; Firestore admin role required. |
| Mapping and Distance Module | Consent-aware coordinates, discovery, distance, ETA and selection. | Implemented; ETA uses external OSRM. |

## 3.3 Functional Requirements

FR1. Register/authenticate users and maintain a Firestore profile.

FR2. Permit valid non-rejected/non-suspended accounts without approval gating.

FR3. Validate profiles, availability, consent and role-specific navigation.

FR4. Create requests with blood group, units, urgency, deadline, hospital, contact and coordinates.

FR5. Filter candidates by account state, compatibility, eligibility, availability, consent, assignment and distance.

FR6. Support donor accept/withdraw/readiness and receiver/admin confirmation.

FR7. Atomically update progress and create one donation/reward outcome per donor/request.

FR8. Maintain recipient notifications, request events and administrative audit records.

FR9. Permit only administrators to execute management operations.

## 3.4 Use Cases

Table 3.2: Representative use cases

| ID | Actor | Precondition | Outcome | Exception |
|---|---|---|---|---|
| UC-01 | User | Valid form/network | Auth account and profile | Duplicate email/invalid data |
| UC-02 | Receiver | Active account | Request created and matching begins | Rate limit/deadline/location error |
| UC-03 | Donor | Eligible and compatible | Assignment reserved | Capacity or active assignment conflict |
| UC-04 | Receiver/Admin | Donor ready | Donation, points and progress recorded | Unauthorized/duplicate action |
| UC-05 | Admin | Role equals admin | Dashboard/action/audit available | Rules reject non-admin |

## 3.5 Non-functional Requirements

Security requires authentication, ownership/role checks, private subcollections, server validation, and no embedded admin password. Usability requires responsive layouts, validation and loading/empty/error states. Reliability requires transactions, idempotency, and graceful network errors. Performance requires pagination and indexed pre-filtering. Privacy requires minimization and opt-in location. Maintainability requires modular services, models, widgets, rules, and tests.

## 3.6 Constraints, Assumptions and Feasibility

Operation depends on Firebase configuration, indexes, quotas, billing and network access. GPS/notifications need permission and coordinates may be inaccurate. Compatibility is a search aid; clinicians must screen and cross-match. Flutter/Dart, Android tooling, Node.js, Firebase CLI/emulators and LaTeX form the software environment. Managed infrastructure makes the prototype technically and economically feasible, while production feasibility requires administrator procedures, privacy/retention policy, clinical partnerships, and local legal review.

# Chapter 4
# System Design

## 4.1 Overall Architecture

Flutter screens/providers form the presentation and state layers; services isolate authentication, Firestore, functions, messaging, storage, location and routing. Firebase supplies identity, persistence and trusted workflows.

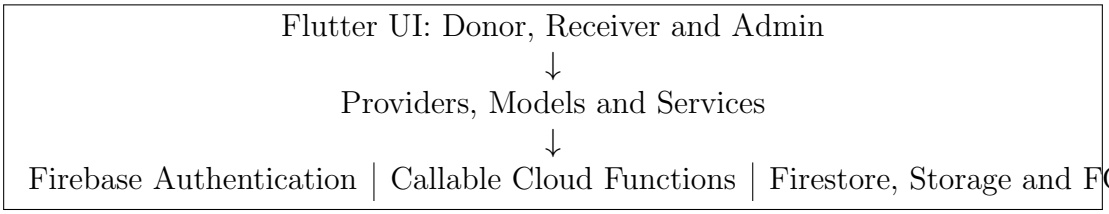



Figure 4.1: Implemented high-level architecture

## 4.2 Interaction and Workflows

Role routing follows authentication. A receiver calls `createBloodRequest`; the function validates fields/rate limits, writes the request and event, and initiates candidate notification. Discovery applies compatibility, active state, availability, eligibility, consent and assignment rules, a latitude-band query, then exact Haversine distance. Acceptance reserves one active request. Readiness and receiver/admin confirmation create a stable donation record, update units and points, and release or complete assignments. SOS creates a linked request rather than an isolated process.

## 4.3 Database Design

Table 4.1: Principal Firestore collections

| Collection | Content | Access/design |
|---|---|---|
| `users` | role, status, blood group, availability, points, eligibility, location | Owner/admin; private contact/devices in subcollections |
| `blood_requests` | receiver, units, urgency, deadline, coordinates, assignments, status | Function-controlled mutations; events subcollection |
| `donations` | participants, date, points, certificate metadata | Participant/admin; stable ID prevents duplicates |
| `sosRequests` | SOS data and linked request | Creator/admin and workflow controlled |
| `notifications` | recipient, content, type and read state | Recipient scoped |
| `rewards`, `blood_drives` | engagement and drive data | Rules constrain writes |
| `reports`, `misuse_reports`, `feedback`, `broadcasts`, `audit_logs` | governance data | Administrative access where appropriate |

## 4.4 UML-style Views

The use-case view assigns profile, availability, acceptance and history to donors; request/SOS creation and confirmation to receivers; and monitoring/moderation to administrators. Shared use cases are authentication, notifications, maps and sign-out.

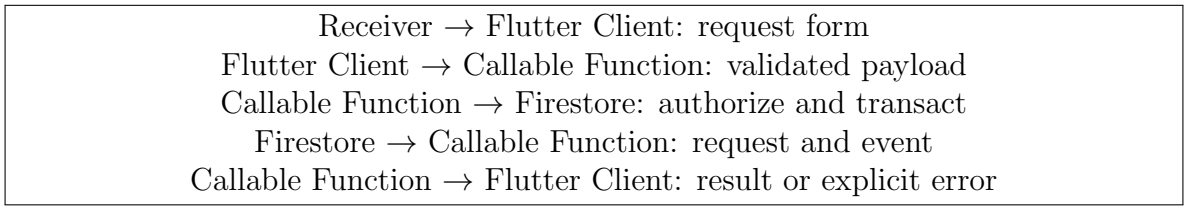



Figure 4.2: Request-creation sequence

The donor activity is authenticate, load candidates, inspect details/distance, accept, optionally withdraw, mark ready, await confirmation, and view history/reward/certificate. Models for users, requests, donations, notifications and rewards map documents to Dart objects; services such as `AuthService`, `GeoService`, `NotificationService` and `CertificateService` isolate external APIs.

## 4.5 Security and Data Flow

Authentication identifies the caller; rules/functions authorize ownership, participation, active account status and admin role. Private contacts and tokens are separated from public profile data.

The client contains project identifiers but no admin password or private signing key. Location sharing is opt-in and matching minimizes returned fields. App Check support is present but production enforcement remains a deployment task.

## 4.6 Chapter Summary

The design separates UI, services and trusted backend transitions, supporting responsive interfaces, least privilege, transactional state and testable policy.

# Chapter 5
# Implementation

## 5.1 Implementation Environment

The client is implemented in Flutter/Dart with Firebase Authentication, Firestore, Functions, Messaging, Storage, Crashlytics and App Check packages. Provider supports state management; `flutter_map`, geolocator, geocoding and `latlong2` support maps; HTTP supports routing; PDF, CSV and sharing packages support documents/exports. Node.js functions implement privileged workflows. Firestore and Storage rules provide a second authorization boundary.

## 5.2 Authentication Module

**Purpose and status.** Registration, login, reset, session restoration, optional email verification and phone-link OTP are implemented in `lib/services/auth_service.dart`, auth screens, validators and `lib/utils/account_policy.dart`. Registration writes status `approved`; login blocks only rejected/suspended accounts. OTP requires Firebase Phone Auth/device configuration.

Forms validate names, email, password, phone and role fields, expose password visibility and loading/error states, and route by Firestore role. Authentication is not authorization: rules and functions recheck status/role. No administrator credential is embedded.

**Require:** Email and password
**Ensure:** Authorized role screen or explicit error
1: Authenticate with Firebase Auth
2: Read the caller's `users` document
3: **if** status is `suspended` or `rejected` **then**
4: sign out and reject
5: **end if**
6: **if** role is donor, receiver, or admin **then**
7: open role navigation
8: **else**
9: reject invalid profile
10: **end if**

Algorithm 5.1: Authenticate and route a user

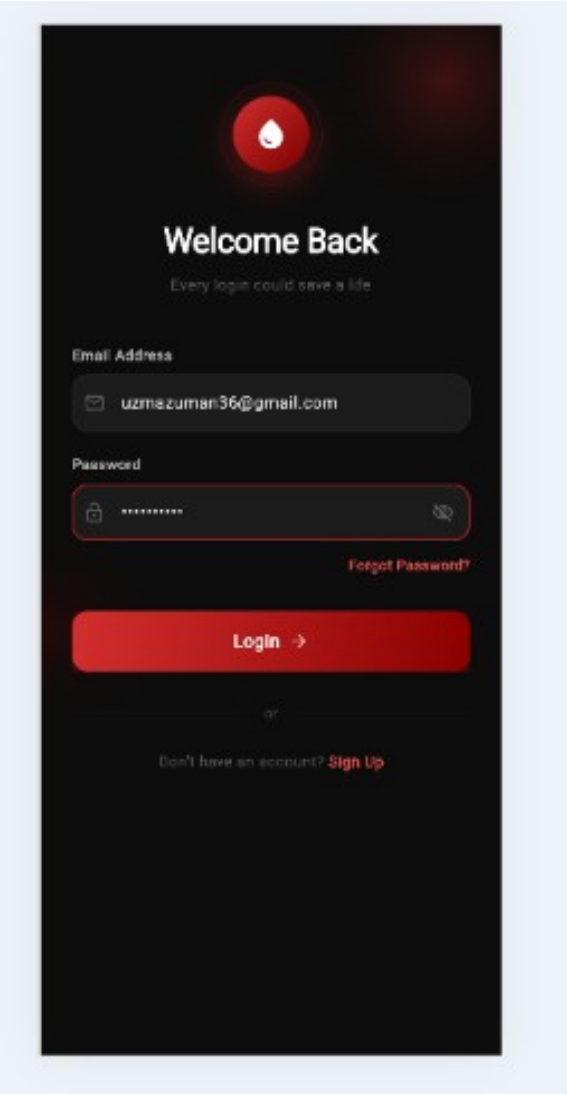



Figure 5.1: Login screen with email/password fields and password-reset access

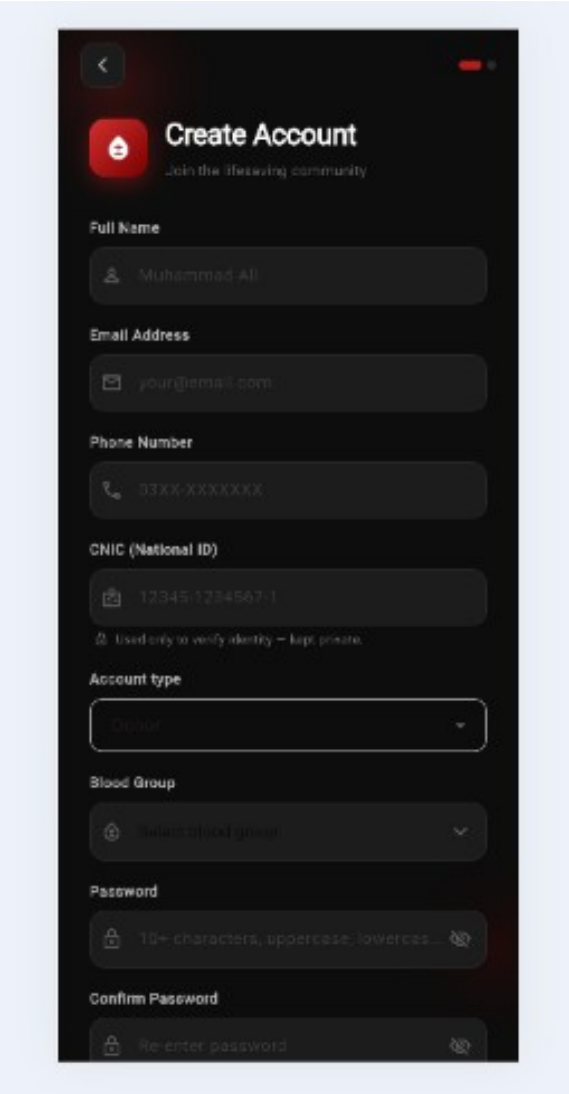



Figure 5.2: Create-account screen with profile, role and blood-group fields

## 5.3 Donor Module

**Purpose and status.** Donor profiles, opt-in location, availability, compatible request discovery, active assignment, readiness, history and eligibility are implemented through donor screens/models, `FirestoreService`, `GeoService`, and request functions. The 90-day interval used by backend policy is an application rule, not clinical clearance.

The backend excludes self, inactive, unavailable, non-consenting, incompatible, ineligible or already-assigned donors. Transactions prevent assignment races; errors preserve state and explain retry requirements.

**Require:** Active donor and open compatible request
**Ensure:** Reserved assignment or rejection
1: Re-read donor and request in a transaction
2: Validate status, availability, eligibility, compatibility and capacity
3: **if** donor has another active request **then**
4: reject
5: **else**
6: add assignment and set `activeRequestId`
7: **end if**

Algorithm 5.2: Accept a donor assignment

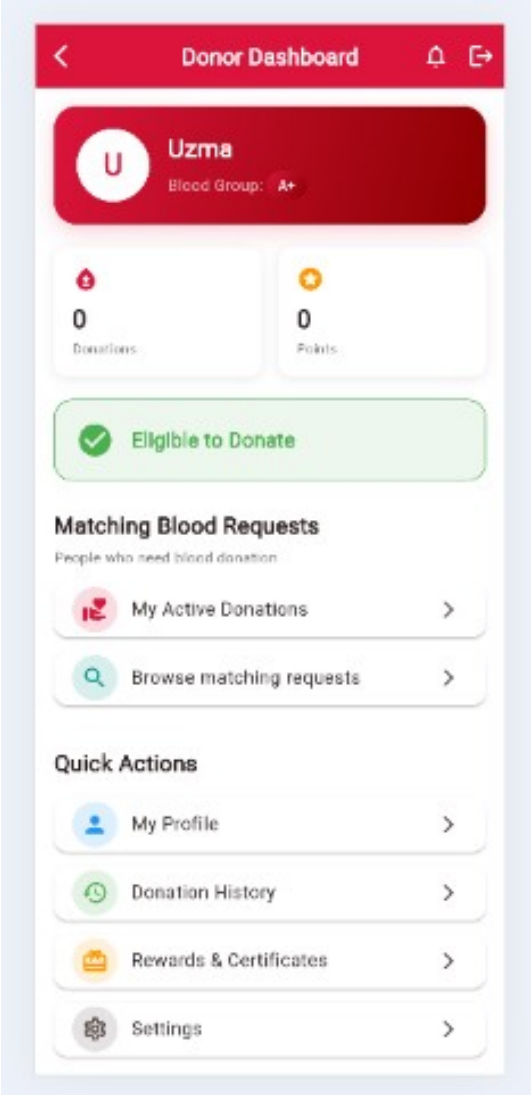



Figure 5.3: Donor dashboard showing donation summary, eligibility and quick actions

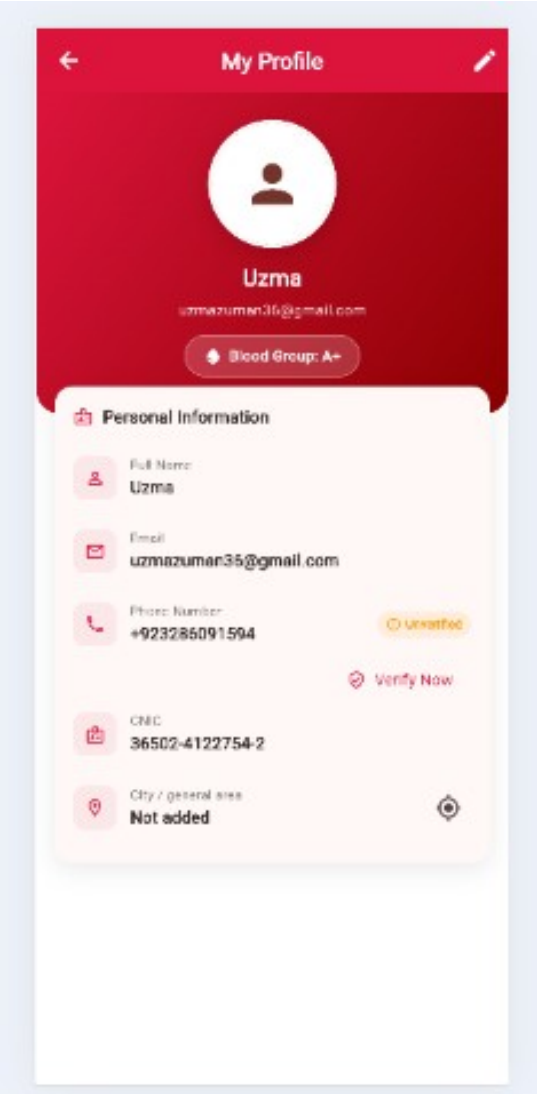



Figure 5.4: Donor profile showing personal information and blood-group details

## 5.4 Receiver Module

**Purpose and status.** Receiver screens call `functions/requestWorkflows.js` to create multi-unit requests, view events/assignments, cancel, contact authorized participants and confirm donations. SOS creates both an SOS record and linked blood request. Inputs include blood group, 1–50 units, urgency, future deadline (maximum 31 days), hospital/contact text and valid coordinates; callable errors cover rate limits and invalid state.

**Require:** Authenticated receiver and valid request fields
**Ensure:** Request/event and matching result
1: Verify active receiver and rate limit
2: Validate group, units, deadline, text and coordinates
3: Store open request and creation event
4: Query compatible nearby donors and create notification work
5: Return request identifier

Algorithm 5.3: Create and process a blood request

The supplied screenshot set does not contain a receiver request or SOS screen. No image is inserted here because the available evidence cannot verify that workflow visually.

## 5.5 Notification Module

**Purpose and status.** `NotificationService`, device-token subcollections and backend notification helpers implement in-app records/read state and FCM delivery for urgent requests, workflow changes, eligibility and drives where configured. Permission denial, absent tokens and network failures are handled without treating push delivery as authoritative state.

**Require:** Recipient, event type and minimized payload
1: Write recipient-scoped notification record
2: Read valid per-device tokens
3: **for all** tokens **do**
4: send through FCM; record/remove invalid token as supported
5: **end for**

Algorithm 5.4: Dispatch a notification

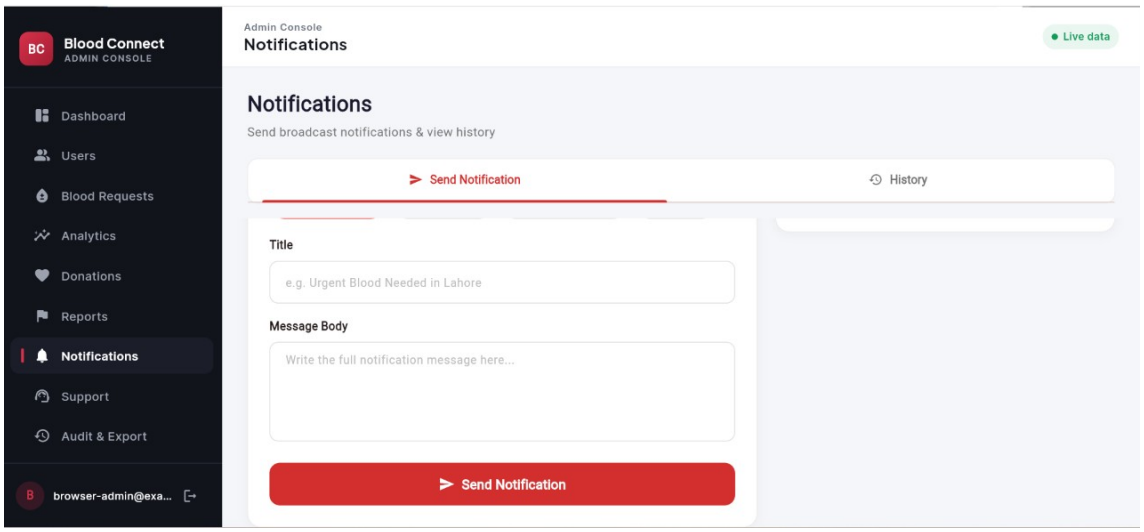



Figure 5.5: Administrative notification composition and history interface

## 5.6 Reward and Certificate Module

**Purpose and status.** Confirmed donations award 50 points and become history/reward inputs. `CertificateService` generates a PDF acknowledgement, includes a QR value tied to the donation identifier, and uploads it to Storage. It does not provide a public QR-verification portal.

**Require:** Ready assignment and authorized receiver/admin
1: Use stable ID `requestId_donorId`
2: **if** donation already exists **then**
3: return idempotent result
4: **end if**
5: Write donation, add points, set next eligibility date
6: Update units/status and release assignment

Algorithm 5.5: Confirm donation and derive reward

The supplied screenshot set does not contain a rewards or certificate screen. The implementation description above is therefore supported by source inspection rather than a fabricated visual.

## 5.7 Admin Module

**Purpose and status.** The responsive admin screens and admin services provide sidebar/drawer navigation, header, aggregate cards, searchable/filterable user lists, status dialogs, reports, misuse handling, request monitoring, broadcasts/exports and audit-oriented actions. Desktop uses persistent navigation; narrower layouts collapse controls. Firebase Auth establishes identity while Firestore role `admin`, rules and callable checks authorize operations.

**Require:** Authenticated caller and requested mutation
1: Read caller profile and verify active `admin` role
2: Validate target, transition, reason and invariants
3: Apply mutation atomically where required
4: Write audit metadata and return sanitized result

Algorithm 5.6: Execute an administrative action

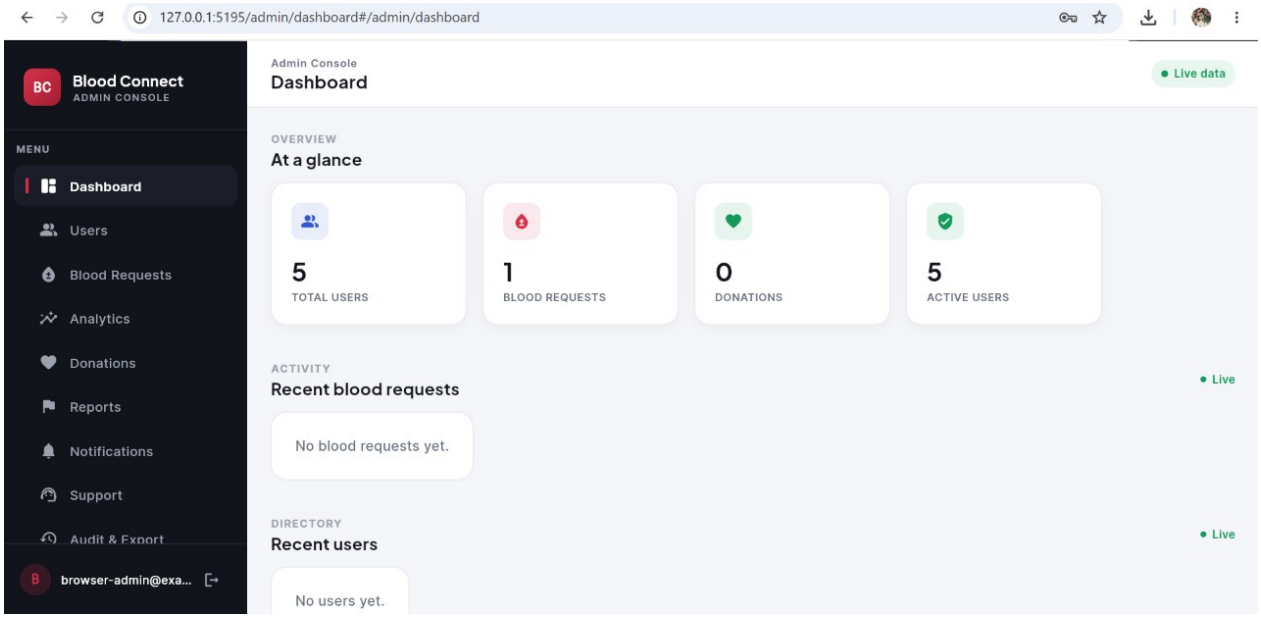



Figure 5.6: Administrative dashboard with summary cards and recent activity

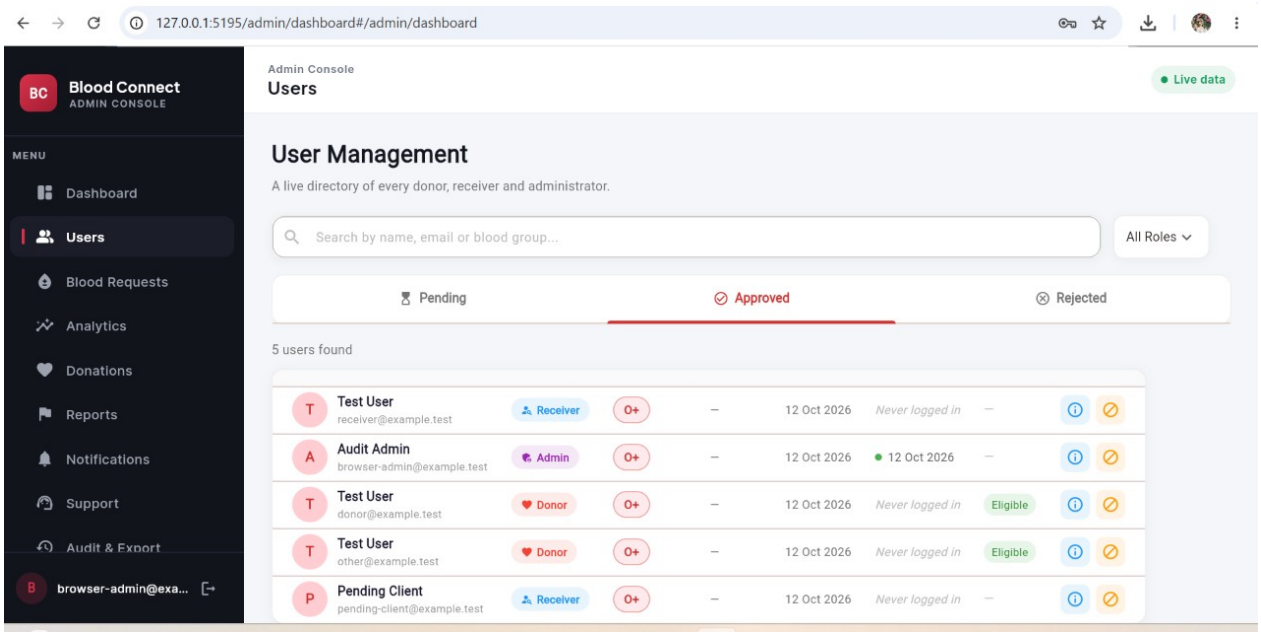



Figure 5.7: Administrative user-management view with approved users and role filters

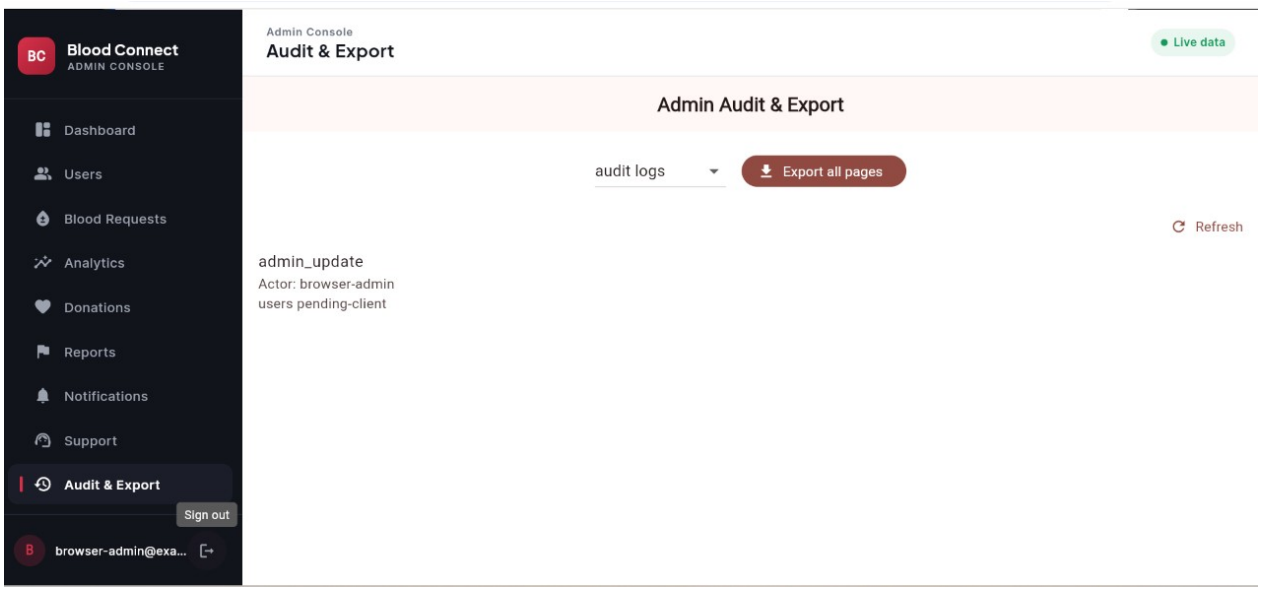



Figure 5.8: Administrative audit and export view showing an audit event

## 5.8 Mapping and Distance Module

**Purpose and status.** `GeoService`, map widgets and backend nearby-donor code implement permission handling, manual/map coordinates, approximate opt-in location, donor search and distance. The server first uses an indexed latitude band, then Haversine distance and policy filters. Client search can expand through 10, 30 and 50 km. OSRM route requests have a timeout and gracefully omit ETA when unavailable.

**Require:** Request coordinates/group and radius
**Ensure:** Minimal ordered candidate list
1: Query approved, available and sharing-enabled donors in latitude bounds
2: **for all** candidates **do**
3: test self, compatibility, eligibility and active assignment
4: compute Haversine distance; retain those within radius
5: **end for**
6: Sort by distance, apply limit/pagination and return minimized fields

Algorithm 5.7: Find nearby eligible donors

The supplied screenshot set does not contain a map or donor-distance screen. Map behaviour remains documented from the implementation source and requires a separate runtime capture for visual evidence.

## 5.9 Implementation Outcome

All seven modules have executable client/backend code, but production behavior remains conditional on Firebase deployment, indexes, credentials, permissions and network services. The

implementation deliberately keeps clinical decisions outside the application.

# Chapter 6
# Software Testing

## 6.1 Objectives, Scope and Integrity

Testing targets validation, policy, widgets, workflow transitions, authorization, concurrency and graceful failure across all seven modules. Evidence was inspected in `test/account_policy_test.dart`, `test/widget_test.dart`, other Flutter tests, and `functions/test`. The repository records earlier successful runs in `docs/verification.md`; however, a fresh `flutter test --no-pub` attempt during this update did not complete and was interrupted. A fresh `npm test` in `functions` stopped before test execution because the local installation could not resolve `firebase-admin/app`. Historical evidence is labelled, and unexecuted scenarios are not promoted to Pass. The account-policy test still expects `pending` to be denied while current policy denies only suspended/rejected; it requires correction and rerun.

## 6.2 Approach and Environment

Unit/widget tests cover validators, eligibility, policy and responsive forms. Firebase Emulator Suite tests cover rules and callable workflows without production data. Physical devices are required for OTP, GPS, push delivery, background handling and complete multi-user evidence.

Table 6.1: Module test cases and evidence

| Test Case ID | Module | Test Scenario | Input | Expected Result | Actual Result | Status | Evidence/Screenshot Reference |
|---|---|---|---|---|---|---|---|
| AUTH-01 | Authentication Module | Valid signup | Unique valid form | Auth user and approved profile | Needs configured backend/device | Not Executed | [SCREENSHOT PLACEHOLDER: Signup] |
| AUTH-02 | Authentication Module | Approval-free login | Valid non-blocked account | Role screen opens | Source allows all except rejected/suspended; fresh suite incomplete | Not Executed | `account_policy.dart` |
| AUTH-03 | Authentication Module | Blocked account | Suspended/rejected status | Access denied | Fresh run incomplete | Not Executed | `account_policy_test.dart` |
| AUTH-04 | Authentication Module | Malformed form | Bad email/weak password | Inline validation | Recorded in prior Flutter run | Pass | `docs/verification.md` |
| AUTH-05 | Authentication Module | Phone OTP | Valid/invalid OTP | Link or explicit error | Needs device/Firebase Phone Auth | Not Executed | [SCREENSHOT PLACEHOLDER: OTP] |
| DON-01 | Donor Module | Eligibility interval | Recent/old dates | Correct eligibility | Recorded prior unit tests | Pass | `widget_test.dart` |
| DON-02 | Donor Module | Accept request | Eligible compatible donor | Atomic assignment | Historical emulator pass | Pass | `functions/test/emulator.test.js` |
| DON-03 | Donor Module | Reject invalid donor | Wrong group/busy donor | No mutation | Historical backend pass | Pass | Backend tests/verification record |
| DON-04 | Donor Module | Responsive UI | Three viewport sizes | No overflow | Visual evidence outstanding | Not Executed | [SCREENSHOT PLACEHOLDER: Donor UI] |
| REC-01 | Receiver Module | Create request | Valid two-unit form | Open request/event | Historical emulator pass | Pass | `emulator.test.js` |
| REC-02 | Receiver Module | Invalid request | Bad units/deadline/location | Reject without write | Fresh execution unavailable | Not Executed | [SCREENSHOT PLACEHOLDER: Validation] |
| REC-03 | Receiver Module | Cancel request | Assigned open request | Cancel and release donors | Historical emulator pass | Pass | `docs/verification.md` |
| REC-04 | Receiver Module | Create SOS | Valid emergency form | Linked SOS/request | Needs deployed callable | Not Executed | [SCREENSHOT PLACEHOLDER: SOS] |
| NOT-01 | Notification Module | Read/empty state | Notification/no data | Correct state/update | Needs app/backend | Not Executed | [SCREENSHOT PLACEHOLDER: Notifications] |
| NOT-02 | Notification Module | Urgent push | Valid token/request | Device receives FCM | Needs deployed FCM/device | Not Executed | [SCREENSHOT PLACEHOLDER: Push] |
| NOT-03 | Notification Module | Denied/stale token | No permission/invalid token | Core workflow survives | Not device-tested | Not Executed | Device log pending |
| RWD-01 | Reward and Certificate Module | Confirm twice | Same request/donor | One donation/award | Historical idempotency pass | Pass | `emulator.test.js` |
| RWD-02 | Reward and Certificate Module | Certificate | Confirmed donation | PDF/upload or clear error | Needs Storage/device | Not Executed | [SCREENSHOT PLACEHOLDER: PDF] |
| RWD-03 | Reward and Certificate Module | Forged reward | Client write | Rules deny | Historical rules pass | Pass | `policy.test.js` |

| Test Case ID | Module | Test Scenario | Input | Expected Result | Actual Result | Status | Evidence/Screenshot Reference |
|---|---|---|---|---|---|---|---|
| ADM-01 | Admin Module | Admin login | Auth user with admin role | Dashboard opens | Needs configured admin | Not Executed | [SCREENSHOT PLACEHOLDER: Admin] |
| ADM-02 | Admin Module | Non-admin action | Donor calls admin function | Deny | Historical authorization pass | Pass | Backend tests |
| ADM-03 | Admin Module | Search/filter | Query/status/role | Correct list/states | Manual evidence outstanding | Not Executed | [SCREENSHOT PLACEHOLDER: Filters] |
| ADM-04 | Admin Module | Status action | Target/reason | Update and audit | Manual backend evidence outstanding | Not Executed | [SCREENSHOT PLACEHOLDER: Action] |
| MAP-01 | Mapping and Distance Module | Radius search | Known coordinates | Ordered in-radius donors | Fresh run incomplete | Not Executed | `nearbyDonors.js` |
| MAP-02 | Mapping and Distance Module | Consent/eligibility | Disabled/ineligible donor | Exclude donor | Historical backend pass | Pass | Backend tests |
| MAP-03 | Mapping and Distance Module | Permission denied | Location denied | Fallback/error shown | Needs device | Not Executed | [SCREENSHOT PLACEHOLDER: Permission] |
| MAP-04 | Mapping and Distance Module | Routing outage | OSRM timeout | Distance remains; ETA omitted | Not network-tested | Not Executed | Device log pending |

## 6.3 Module Assessments

### 6.3.1 Authentication Module

Positive and negative cases must cover credentials, blocked states and non-admin access. The stale pending-status assertion is a known maintenance issue.

### 6.3.2 Donor Module

Historical policy/emulator evidence covers compatibility, eligibility, assignment and forgery resistance; visual evidence remains.

### 6.3.3 Receiver Module

Creation, cancellation and concurrency have historical emulator evidence; SOS propagation needs deployment testing.

### 6.3.4 Notification Module

Token/UI logic exists, but foreground/background receipt and permissions require devices.

### 6.3.5 Reward and Certificate Module

Idempotency has historical backend evidence; PDF upload/share and QR content require manual evidence.

### 6.3.6 Admin Module

Rules/functions enforce roles; responsive UI, filters, dialogs, exports and audit visibility need configured-admin testing.

### 6.3.7 Mapping and Distance Module

Haversine policy exists; GPS accuracy, permission handling and OSRM ETA require field testing.

## 6.4 Visual Test Evidence

The following figures document the supplied application states used for visual inspection. They are evidence of the rendered screens and workflow checkpoints, not a substitute for automated execution or device-level testing. The observed values are reproduced exactly as shown in the screenshots.

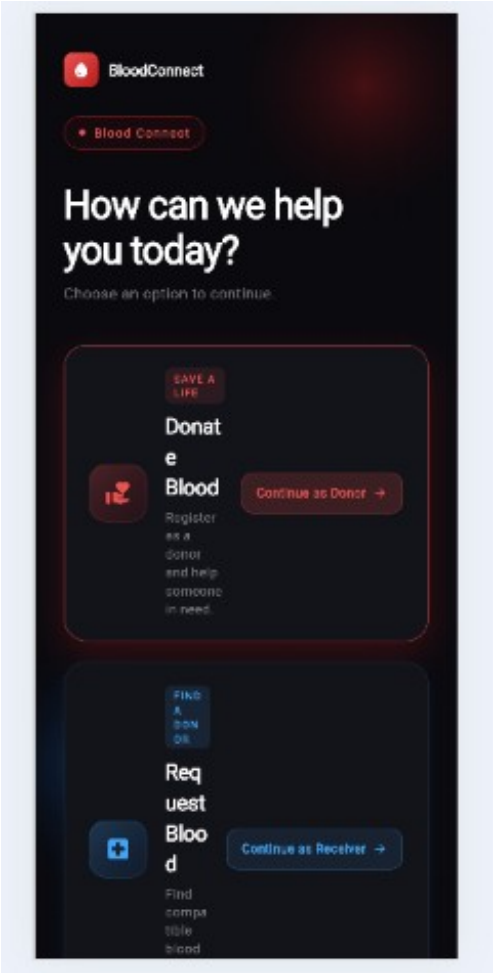



Figure 6.1: Role-selection screen used to inspect donor and receiver navigation entry points.

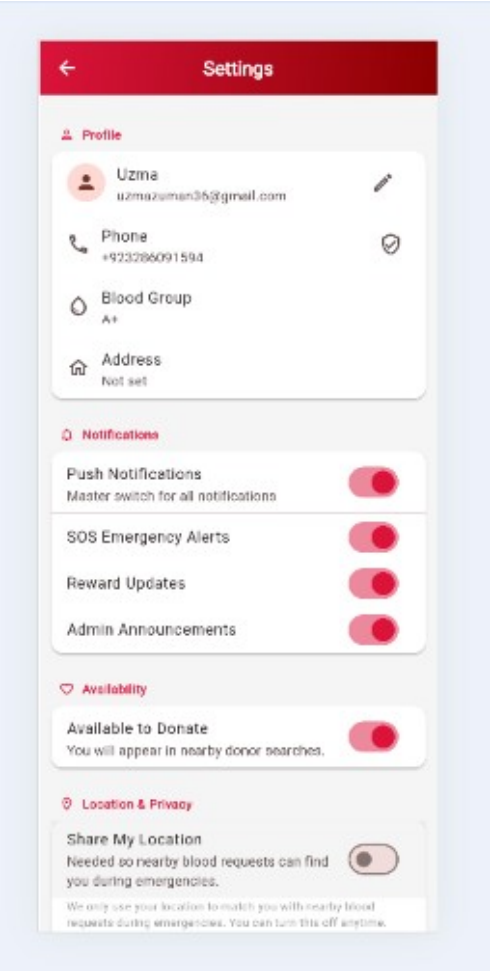



Figure 6.2: Settings screen used to inspect profile data, notification controls and donation availability.

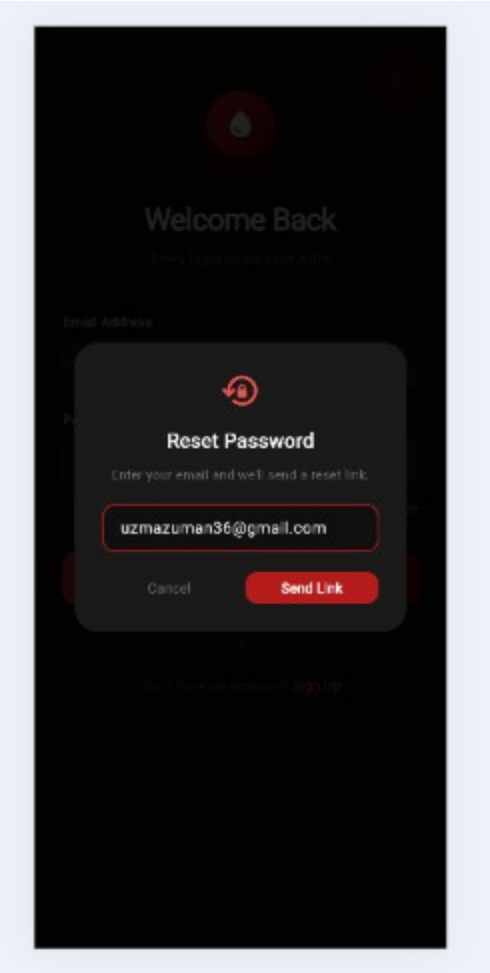



Figure 6.3: Reset-password dialog used to inspect input presentation and the reset-link action.

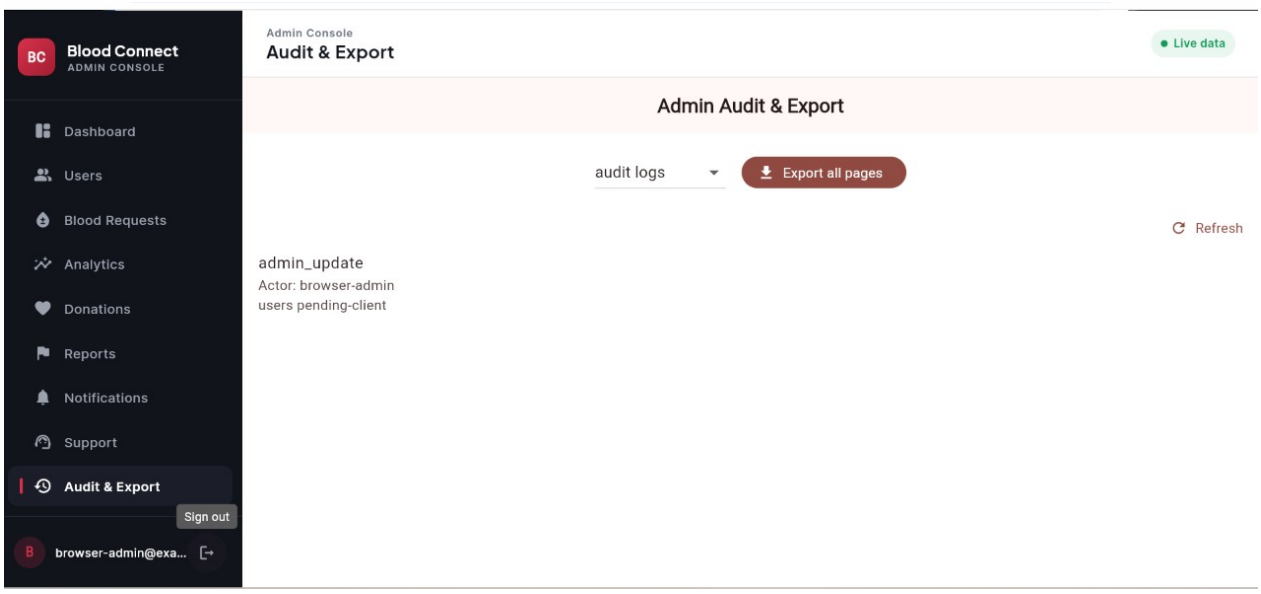



Figure 6.4: Admin audit view used to inspect audit-log visibility and export controls.

The supplied email-link and email-verified images provide evidence of the external Firebase email-verification step. They do not prove delivery reliability for all accounts; that remains a device and backend test requirement. Similarly, the donor dashboard and administrator screenshots support responsive visual review, while the test matrix correctly leaves automated and production-dependent cases as *Not Executed* where no execution record was supplied.

## 6.5 Security, Limitations and Summary

Emulator tests target owner/participant/admin boundaries, forged writes, transitions, cancellation and concurrent confirmation. Production rules, indexes, App Check and deployed functions still require comparison before release. No fresh full Flutter, backend, or physical-device test run completed during this edit: the Flutter process stalled and the Node test process lacked its installed Firebase Admin dependency. Remaining evidence includes dependency restoration, current suite reruns, two-device flows, OTP, FCM, GPS, certificates, admin actions and responsive screenshots; Table 6.1 uses *Not Executed* where current evidence is absent.

# Chapter 7
# Conclusion and Future Work

## 7.1 Conclusion

**Guidelines (300–500 words):** Summarize:

- Problem addressed
- System solution
- Key achievements
- Evaluation results
- Overall contribution

Write your conclusion here.

## 7.2 Future Work

**Guidelines (200–300 words):** Mention:

- System improvements
- Scalability enhancements
- AI/ML improvements
- Security enhancements
- Additional modules

Write future work here.

## 7.3 Limitations

Briefly mention system limitations:

- Data limitations
- Infrastructure limitations
- Model constraints

# Bibliography

[1] World Health Organization, "Blood safety and availability," accessed 10 October 2026. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability

[2] ——, "Blood products: Why should i donate blood?" accessed 10 October 2026. [Online]. Available: https://www.who.int/news-room/questions-and-answers/item/blood-products-why-should-i-donate-blood

[3] L. Li, M. Valero, R. Keyser, O. Ukuku, and K. Zheng, "Mobile applications for encouraging blood donation: A systematic review and case study," *Digital Health*, vol. 9, 2023.

[4] American Red Cross, "Blood donor app," accessed 10 October 2026. [Online]. Available: https://www.redcrossblood.org/blood-donor-app.html

# Appendices

## Appendix A - Turnitin Similarity Report

This appendix contains the official Turnitin Similarity Report generated for the final submitted version of this dissertation.

**Plagiarism Report Screenshot:**

# Appendix B - AI Writing Detection Report

This appendix contains the AI Writing Detection Report generated by Turnitin (if applicable), verifying compliance with institutional academic integrity guidelines regarding AI-assisted content.

**AI Detection Report Screenshot:**

# Appendix C – Source Code

This section provides details about the project source code repository.

**Guidelines for Students:**

- Upload your complete project source code to a version control platform such as GitHub, GitLab, or Bitbucket.
- Ensure the repository includes:
  - All source files
  - Database scripts (if applicable)
  - Model training and testing code (for ML projects)
  - Frontend and backend code (for web/mobile projects)
  - README file with setup instructions
- The repository must be properly structured with meaningful folder names.
- Remove unnecessary files (temporary files, cache files, datasets if restricted).
- If the repository is private, make the repository public before final submission.
- Paste the final repository link below.

**Repository Link:**

`https://github.com/usmanch-15/blood-bank`

**Additional Notes:**

- Mention branch name if different from `main`.
- Mention if any third-party APIs require API keys.
- Mention hardware requirements (if ML/GPU-based project).